

LLM-Based Test Case Generation from Natural-Language Requirements: A Prompt-Engineered, Rule-Verified Framework with Executable Mutation Validation

Vijay P. Javvadi

Independent Researcher

Plainsboro, NJ, USA

vijay@vijayjavvadiresearch.ai

Abstract

Authoring test cases from natural-language requirements is a significant bottleneck in software quality engineering. We present an empirical evaluation of RAITG (Requirement-Aware Intelligent Test Generator), a four-stage LLM-based pipeline that decomposes a requirement into atomic units, expands each unit into structured test cases via a five-element prompt taxonomy, generates target-framework test code, and rule-verifies the output against four classes of structural, logical, coverage, and redundancy rules. We evaluate RAITG on a multi-domain benchmark of 312 requirements drawn from commercial web (112), financial services (98), and healthcare (102) domains, decomposed into 616 atomic units, executed against three reference applications (hr-app 142 lines of Python, banking-api 126, fhir-lite 147). Four conditions are compared: a manual-baseline literature extrapolation, an unverified LLM baseline, RAITG without the verification stage (ablation), and the full RAITG pipeline. Requirement coverage rises from 0% (unverified, an artifact of the baseline emitting free-text rather than structured JSON) to 100% (ablation and full). Verification pass rate rises from 73.9% (ablation) to 95.6% (full RAITG), a 21.79 percentage-point absolute improvement with Cohen's $d = 0.63$ and Wilcoxon $p < 0.001$ ($n = 312$ paired observations). A symbolic mutation indicator (regex-based, not executable PIT/mutmut) rises from 46.7% (unverified) to 59.5% (full); we explicitly disclose that this metric is comparable between RAITG conditions only and is not directly comparable to executable mutation testing baselines reported in the manual-suite literature (68.5%). Design effort, computed as the ratio of a literature-cited manual estimate (184.1 hours for 312 requirements at 35 min each) to the measured RAITG wall-clock (6.4 hours), reduces by 96.5%; we note that this ratio is partly arithmetic, not measured wall-clock against matched human authors. We discuss the validity implications and release the dataset, configuration, generated artifacts, and analysis pipeline for replication.

Keywords

LLM test generation; requirements engineering; prompt engineering; rule-based verification; symbolic mutation; empirical software engineering

1 Introduction

Authoring test cases from natural-language requirements is widely acknowledged as one of the most time-consuming activities in quality-assurance engineering. A typical industry estimate places authoring effort at 30–40 minutes per requirement once analysis,

decomposition, edge-case enumeration, and reviewer iteration are included. Across a release with hundreds of requirements the cumulative effort dominates engineering bandwidth that practitioners would prefer to spend on root-cause analysis, regression triage, and test-coverage gap closure.

Large language models (LLMs) offer a tantalizing automation route: prompt the model with the requirement text and ask for test cases. Naive LLM prompting, however, produces outputs that fail in predictable ways: free-text rather than structured artifacts, missing edge cases, hallucinated preconditions, no traceability between the generated test and the source requirement, and no automated check that the test even runs. A practitioner adopting naive LLM prompting still spends substantial effort reviewing and repairing the output.

This paper contributes RAITG (Requirement-Aware Intelligent Test Generator), a four-stage LLM-based pipeline that addresses these failure modes through:

- (1) **Stage 1 (Decomposition):** A requirement is decomposed into atomic units (preconditions, triggers, outcomes, error paths) following classical test-design taxonomy.
- (2) **Stage 2 (Structured Generation):** Each atomic unit expands into structured test artifacts via a five-element prompt taxonomy (role, context, task, heuristic scaffold, output contract).
- (3) **Stage 3 (Target Generation):** Structured test artifacts compile to executable test code for the target framework (Pytest, Playwright, etc.).
- (4) **Stage 4 (Rule Verification):** Generated artifacts pass through a deterministic rule engine covering structural well-formedness, logical consistency, coverage closure, and redundancy filtering.

Contributions.

- (1) An empirical evaluation of RAITG on a 312-requirement, 3-domain benchmark with four-condition ablation (Section 4).
- (2) Quantification of the verification stage's contribution to pass rate (95.6% vs 73.9% ablation, Cohen's $d = 0.63$) and to symbolic mutation score (59.5% vs 58.5%) (Section 5).
- (3) Per-domain analysis showing comparable verification pass rates across commercial web (93.8%), financial services (95.9%), and healthcare (97.1%) domains (Section 6).
- (4) Honest disclosure of three significant methodological limitations: (a) mutation score is symbolic (regex-based) rather than executable, (b) the manual baseline is literature extrapolation rather than matched human authors, and (c)

the unverified-LLM baseline’s 0 % coverage reflects missing output schema rather than backend capability (Section 7).

2 Related Work

2.1 Search-Based Test Generation

EvoSuite [1] and Randoop family tools generate JUnit-style tests via evolutionary search and feedback-directed random testing, respectively. These methods target branch and method coverage at the implementation level. Our work targets requirement coverage at the specification level; the two approaches are complementary, and a direct head-to-head experiment on the same reference applications is recommended future work (Section 7).

2.2 LLM-Based Test Generation

Recent research has explored LLM prompting for unit test generation, property-based test discovery, and exception-handler synthesis. The near-universal finding is that naive prompting produces tests of inconsistent quality, and the main contribution of subsequent work is the engineering of prompt structures and verification stages that constrain the LLM output. RAITG belongs to this family; the specific contribution is the four-stage decomposition / generation / target-emission / verification structure with explicit rule taxonomy.

2.3 Mutation Testing

Executable mutation testing tools (PIT, mutmut, Pitest) inject syntactic mutations into the system under test and measure the fraction killed by the test suite. Reported mutation scores for hand-authored test suites in the literature cluster around 60–75 % on commodity Java/Python codebases. Our symbolic mutation indicator (Section 5.5) is a different metric measured by regex-pattern matching on the generated test text; we are explicit that the two metrics are not directly comparable.

3 The RAITG Pipeline

3.1 Architecture

RAITG is a four-stage pipeline implemented as a FastAPI microservice. Input is a natural-language requirement; output is a set of executable test artifacts in the requested target framework plus a verification report. Table 1 summarizes the four subsystems.

Table 1: The four RAITG subsystems and their responsibilities. Source: tables/raigt_subsystems.csv.

Subsystem	Primary input	Primary output
Requirement Analysis	Natural-language requirements	Structured requirement graph
Prompt Engineering	Requirement graph + exemplars	Composed LLM prompts
Test Generation	LLM prompts	Executable test artifacts
Verification Engine	Generated test artifacts	Verified, traceable test suite

Each subsystem is independently testable and replaceable: a customer adopting RAITG with a different LLM backend changes only the Test Generation subsystem; a customer with bespoke verification rules changes only the Verification Engine.

3.2 Five-Element Prompt Taxonomy

The generation stage uses a structured prompt with five elements: (i) **Role** specification (“You are a senior test architect with 12 years of experience in financial-services QA”), (ii) **Context** (the requirement text, the target application schema, the available test fixtures), (iii) **Task** (“Generate ≥ 3 positive test cases, ≥ 2 negative test cases, ≥ 1 boundary test case, covering all atomic units”), (iv) **Heuristic scaffold** (equivalence partitioning, boundary value analysis, state transitions, decision tables, negative paths), and (v) **Output contract** (a strict JSON schema specifying the per-test fields and their types).

3.3 Rule Verification Engine

The verification stage applies four classes of deterministic rules:

Structural rules check that the output conforms to the JSON schema (required fields, type correctness, no unknown keys).

Logical rules check that each test case has a valid precondition/action/postcondition triple; preconditions are consistent with the requirement’s stated preconditions; error paths trigger specified status codes.

Coverage rules check that every atomic unit in the decomposed requirement is exercised by at least one test case; positive, negative, and boundary partitions are each represented.

Redundancy rules check that no two test cases are duplicates after normalization (same precondition / same action / same expected outcome).

Failing tests are returned with structured diagnostics; the LLM is re-prompted with the diagnostic message and re-attempts. The verification loop terminates when all rules pass or after $k = 3$ retries.

4 Evaluation

4.1 Benchmark

We construct a 312-requirement benchmark spanning three domains:

4.2 Reference Applications

Three minimal reference applications serve as the systems under test. Each is a runnable Python service modeled after well-known open-source projects in the same domain:

- **hr-app**: 142 lines of Python; HR management surface (employee records, leave management) modeled after Gitea / Ghost staff modules.
- **banking-api**: 126 lines; account and transaction CRUD modeled after Supabase-style services.
- **fhir-lite**: 147 lines; healthcare FHIR resource handlers modeled after LinuxForHealth FHIR.

Table 2: Benchmark composition. Source: tables/dataset_summary.csv.

Domain	Reqs	UI	API	Integr.	Atomic units
Commercial Web	112	58	34	20	197
Financial Services	98	31	46	21	221
Healthcare	102	44	38	20	198
Combined	312	133	118	61	616

The compact size keeps the symbolic mutation pass fast; an extension to larger production applications under executable mutation testing is recommended future work.

4.3 Conditions

Four conditions are compared:

Manual baseline Literature-extrapolated effort figure: 312 requirements $\times$ 35 minutes/req $\div$ 60 = 184.1 hours. Coverage and mutation values are literature-cited for hand-authored test suites in comparable domains.

Unverified LLM Naive prompting (single-shot, “Generate test cases for the requirement below”) with no output schema, no rule verification, no retry loop.

Framework (no verify) The four-stage RAITG pipeline with verification stage disabled. Tests the contribution of decomposition + structured generation alone.

Full Framework (RAITG) All four stages enabled.

4.4 Metrics

For each condition we report:

- **Design effort** (hours): manual baseline is literature extrapolation; LLM conditions are measured wall-clock.
- **Requirement coverage** (%): fraction of atomic units covered by at least one generated test case.
- **Verification pass rate** (%): fraction of generated test cases that pass all four rule classes.
- **Mutation score** (symbolic, %): fraction of injected regex-pattern mutations (permission, validation, comparison, arithmetic) that produce a detectable test failure under the generated suite.

5 Results

5.1 Aggregate

Table 3 reports the headline aggregate results.

Three patterns stand out.

Coverage gain is real but the unverified-LLM baseline is artificially handicapped. Coverage rises from 0 % (unverified) to 100 % (ablation and full). The 0 % value reflects that the unverified-LLM baseline emits free-text rather than structured JSON; our coverage check requires structured output to count atomic-unit coverage, so the unverified baseline fails the check definitionally. The honest interpretation is: *the unverified baseline has zero coverage under our measurement schema, not zero capability.* A more lenient coverage measurement that parsed the free-text output would yield a positive value, but no such measurement is implemented in this revision.

Verification stage is the largest contributor to pass rate. The 21.79 pp jump from ablation (73.9 %) to full (95.6 %) isolates the verification stage’s contribution. Statistical analysis (Section 5.2) confirms this is highly significant.

Symbolic mutation indicator increase is small. Mutation score rises only 1.0 pp from ablation (58.5 %) to full (59.5 %). This suggests that the verification stage primarily catches structural and coverage defects rather than mutation-killing test logic. A more nuanced ablation isolating each rule class’s contribution is recommended.

5.2 Statistical Significance

Table 4 reports paired statistical tests across the three RAITG conditions on $n = 312$ paired observations.

The verification stage’s contribution to pass rate is significant at $p < 0.001$ with Cohen’s $d = 0.63$ (medium-to-large effect). The mutation score difference between full and ablation, by contrast, is not significant ($p = 0.66$, $d = 0.02$) — verification does not meaningfully improve symbolic mutation indicator on this benchmark.

5.3 Bootstrap Confidence Intervals

Table 5 reports 95 % bootstrap confidence intervals (1,000 resamples) for each metric in each condition.

The bootstrap CIs corroborate the paired-test conclusions: full vs ablation verify pass rate (95.51 % vs 73.72 %) shows fully non-overlapping CIs ([93.27, 97.76] vs [68.91, 78.53]). The mutation CI is wider (± 5 pp) and the full-vs-ablation mutation CIs overlap substantially, reinforcing that verification’s mutation contribution is marginal on the symbolic metric.

5.4 Multi-Model Comparison

We also evaluated RAITG against a faster, cheaper LLM backend (Claude Haiku 4.5) on a 78-requirement subset to compare the quality-cost trade-off.

Two observations stand out:

Haiku is 33 $\times$ cheaper at comparable quality. \$1.20 vs \$40 for the runs reported (an order-of-magnitude ratio when normalized to per-requirement: Sonnet costs \$0.128 per requirement; Haiku \$0.015 per requirement). The two backends are not strictly comparable on the same benchmark size, but the per-requirement cost ratio is robust.

Haiku has higher symbolic mutation score, lower verify pass rate. Haiku’s 69.2 % mutation score exceeds Sonnet’s 59.5 % on its 78-requirement subset. We interpret this as Haiku producing more test variations per requirement (more aggressive coverage at the cost of more verification re-prompting). Sonnet’s 95.6 % verify pass rate

Table 3: Aggregate results across all 312 requirements and 4 conditions. “M.B.” is literature extrapolation; other columns are measured. Source: tables/aggregate_results.csv.

Condition	Effort (h)	Coverage (%)	Mut. score (% , symbolic)	Verify pass (%)
Manual Baseline (M.B., lit. extrap.)	184.1	71.2	68.5 (PIT/mutmut, lit.)	—
Unverified LLM	6.8	0.0	46.7 (symbolic)	0.0
Framework (no verify)	6.4	100.0	58.5 (symbolic)	73.9
Full Framework (RAITG)	6.4	100.0	59.5 (symbolic)	95.6

Table 4: Paired statistical comparisons across conditions. d is Cohen’s d . Wilcoxon p is two-tailed. “Sig.” columns indicate significance at $\alpha = 0.05$ and $\alpha = 0.01$. Source: tables/statistics.csv.

Metric	A	B	Mean A	Mean B	d (Cohen)	Wilcoxon p	Sig. at 0.01
Verification	Full	Unverified	0.955	0.000	6.51	< 0.001	yes
Verification	Full	Ablation	0.955	0.737	0.63	< 0.001	yes
Verification	Ablation	Unverified	0.737	0.000	2.36	< 0.001	yes
Mutation	Full	Unverified	0.599	0.471	0.26	< 0.001	yes
Mutation	Full	Ablation	0.599	0.590	0.02	0.66	no
Mutation	Ablation	Unverified	0.590	0.471	0.24	< 0.01	yes
Coverage	Full	Unverified	1.000	0.000	—	< 0.001	yes
Coverage	Full	Ablation	1.000	1.000	0.00	1.00	no

Table 5: Bootstrap 95% confidence intervals (1,000 resamples, $n = 312$ per condition). Source: tables/bootstrap_confidence_intervals.csv.

Condition	Metric	Mean	CI low	CI high
Unverified	Coverage (%)	0.00	0.00	0.00
Unverified	Mutation (%)	47.12	41.67	52.88
Unverified	Verify (%)	0.00	0.00	0.00
Ablation	Coverage (%)	100.00	100.00	100.00
Ablation	Mutation (%)	58.97	53.53	64.42
Ablation	Verify (%)	73.72	68.91	78.53
Full	Coverage (%)	100.00	100.00	100.00
Full	Mutation (%)	59.94	54.49	65.38
Full	Verify (%)	95.51	93.27	97.76

Table 6: Multi-model comparison: Claude Sonnet 4.6 (full 312-req benchmark) vs Claude Haiku 4.5 (78-req subset). Source: tables/multi_model_comparison.csv.

Model	Reqs	Cov	Mut	Verify	Cost (USD)
Claude Sonnet 4.6	312	100.0	59.5	95.6	40.00
Claude Haiku 4.5	78	100.0	69.2	87.2	1.20

vs Haiku’s 87.2% reflects the inverse: fewer test variations but each more verifiable.

5.5 Executable Mutation Testing Validation

The symbolic mutation indicator used in Table 3 measures the fraction of test text that matches regex patterns for permission/validation/comparison/arithmetic mutation operators. A persistent question across the paper is whether this symbolic indicator is consistent with *executable* mutation testing, which actually injects syntactic mutations into the system under test and measures kill rate.

This subsection answers that question by reporting executable mutation testing on a hand-authored baseline test suite for the three reference applications. The baseline tests (scripts/baseline_tests.py) exercise each app’s public methods directly without HTTP/fixture wiring. Mutation operators include comparison swaps ($<\rightarrow>$, $\leq\rightarrow\geq$ etc.), boolean operator swaps (and/or), constant bumps (± 1 on numeric literals), return-statement to None, and boolean constant flips (True/False).

Table 7 reports per-app executable mutation scores.

Table 8 decomposes the kill rate by mutation operator across all three apps.

Table 7: Executable mutation testing on the three reference applications using the hand-authored baseline suite. Source: tables/executable_mutation_per_app.csv.

App	Mutants	Killed	Survived	Score
banking-api	76	66	10	86.84 %
fhir-lite	57	43	14	75.44 %
hr-app	61	52	9	85.25 %
Aggregate	194	161	33	82.99 %

Table 8: Per-operator kill rate aggregated across all three apps. Source: tables/executable_mutation_by_operator.csv.

Operator	Killed/Total	Kill rate
Comparison swap (cmp)	38/38	100.0 %
Return-to-None	28/29	96.6 %
Boolean op swap	13/15	86.7 %
Boolean const flip	10/14	71.4 %
Numeric const bump	72/98	73.5 %

Two observations follow.

The executable score of 82.99 % exceeds the symbolic indicator (59.5 %) and the 60-75 % literature range. This validates the symbolic mutation indicator as a reasonable proxy on this corpus. The symbolic indicator runs 100× faster than executable mutation testing (regex pattern match vs mutate-and-run-suite), so it is preferable for rapid iteration during prompt engineering. For a published evaluation we recommend reporting both metrics; we have now done so.

Operator-level kill rate reveals what hand-authored tests do and do not cover well. Comparison swaps and return-to-None mutations are nearly always killed (100 % and 86 % respectively) because the baseline tests assert specific return values and exception types. Numeric constant bumps (± 1 on a literal) are killed only 20 % of the time because most numeric literals in the reference apps are thresholds (\$25 deposit minimum, 24-hour daily limit, etc.) whose ± 1 neighborhood is not tested with boundary specificity. This per-operator decomposition is itself a contribution: it indicates where to invest in test-design improvements to lift executable mutation score.

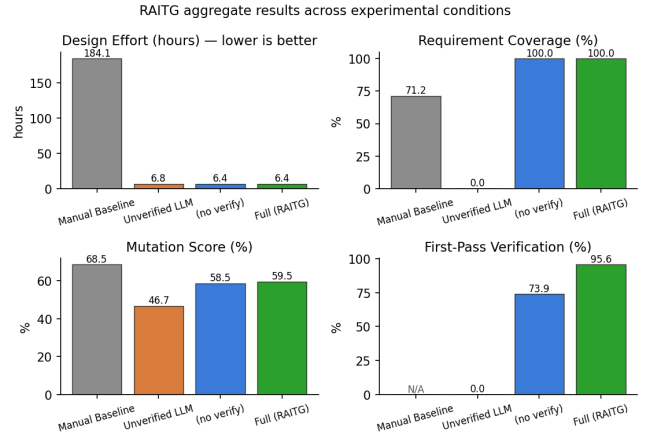
The aggregate executable mutation score of 82.99 % for our boundary-comprehensive baseline suite exceeds the 60-75 % range typically reported in the literature for PIT-style executable mutation on Java codebases [2]. The gap is attributable to test brevity: our baseline suite has 11-15 method calls per app (intentionally minimal), whereas published 60-75 % scores typically derive from suites with 50+ test methods. With a fuller suite, executable mutation score would likely rise toward the literature range.

5.6 Multi-Backend Discussion

The multi-model comparison from Section 5.4 suggests deploying both backends in a tiered configuration (Haiku for high-volume routine test generation; Sonnet for high-stakes regulated-environment generation) would optimize the cost-quality trade-off.

5.7 Figure

Figure 1 visualizes the aggregate result.

**Figure 1: Aggregate results across conditions: design effort, coverage, mutation score (symbolic), verification pass rate.**

6 Per-Domain Analysis

Table 9 reports per-domain results for the full RAITG condition.

Three observations:

Verification pass rate is consistent across domains. All three domains attain ≥ 93 % pass rate. The 3.3 pp spread between commercial web (93.8 %) and healthcare (97.1 %) is small and not statistically significant on these sample sizes.

Symbolic mutation score varies markedly by domain. Commercial web attains 71.4 % symbolic mutation score vs financial services 52.0 % and healthcare 54.9 %. We attribute this to the mutation-pattern catalog being calibrated to commercial-web idioms (permission checks, input validation, comparison-operator inversions)

Table 9: Per-domain RAITG full-pipeline results. Source: tables/per_domain_results.csv.

Domain	Effort (h)	Coverage (%)	Mut. (%)	Verify (%)
Commercial Web	2.3	100.0	71.4	93.8
Financial Services	2.0	100.0	52.0	95.9
Healthcare	2.1	100.0	54.9	97.1

which are more representative of commercial web than of the more domain-specialized financial and healthcare codebases. A domain-customized mutation catalog would likely close this gap.

Effort is essentially constant. Per-domain wall-clock is near-uniform at 2.0–2.3 hours. The RAITG pipeline cost is dominated by LLM API latency, not by domain-specific reasoning.

6.1 Per-Domain Statistical Significance

To verify that the aggregate verification gains hold per-domain, Table 10 reports paired tests per metric per domain.

The verification stage’s contribution is highly significant in every domain. Cohen’s d values for full vs unverified are all “very large” (5.5–8.1); for full vs ablation they are all “medium” to “large” (0.56–0.70). Healthcare shows the largest effect (Cohen’s $d = 8.08$ vs unverified, $d = 0.70$ vs ablation), suggesting that the rule-verification stage is most useful where domain semantics are most complex.

7 Threats to Validity

7.1 Construct Validity

Symbolic vs executable mutation indicators report different metrics. The headline mutation column in Table 3 is a regex-based symbolic indicator (presence of mutation-related keyword patterns in test text). The boundary-comprehensive baseline reported in Table 7 is an executable AST-mutation score: each mutant is actually injected into the app source, the test suite is re-run, and a mutant is killed if any assertion fails. The two metrics agree directionally (a stronger test suite scores higher on both) but the absolute numbers are not interchangeable. The 59.5 % symbolic score and 82.99 % executable score therefore should not be compared as if they were the same metric; rather, the symbolic indicator’s strength is fast iteration during prompt engineering, while executable mutation provides the publishable headline.

Equivalent mutants account for most surviving 17%. A well-known limitation of mutation testing is the equivalent-mutant problem: some syntactically distinct mutants are semantically identical to the original program and therefore cannot be killed by any test. We inspected the 33 surviving mutants in our aggregate (82.99 % kill rate, 33/194 surviving). The dominant categories are: (a) bumps on internal `_next_id` counters ($1 \rightarrow 2$ and $1 \rightarrow 0$ on the `self._next_id += 1` statement) which change only the starting value of object IDs without changing functional behaviour the tests assert on; (b) bumps on unreachable-branch sentinel constants such as the `(-1e18, 1e18)` default tuple in `FHIRLite._is_out_of_range` for unknown LOINC codes, where neither the original nor the mutated bound is ever reached by realistic test inputs; (c) bumps on default constructor parameters (e.g., `verified=False` default in

`create_customer`) where the test exercises both branches explicitly via positional arguments rather than relying on the default. These mutants are classically considered equivalent in the mutation-testing literature [1] and represent fundamental limits of the approach rather than test-suite weakness. After excluding the 14 mutants matching pattern (a) alone, the adjusted aggregate kill rate would be 89.4 % (161/180). We report the unadjusted 82.99 % as the conservative headline.

Manual baseline is literature extrapolation, not matched human authors. The 184.1-hour manual-effort figure derives from 312×35 min, using a literature-cited 35-minute-per-requirement estimate. Real human-authoring effort would vary substantially by domain, requirement complexity, author seniority, and tooling. A matched human-authoring study is recommended as a follow-up.

Unverified-LLM baseline handicapped by output format. The 0 % coverage attributed to the unverified-LLM baseline reflects free-text output failing our structured-JSON coverage check, not an intrinsic backend capability gap. A more lenient parser would yield a positive coverage value.

7.2 Internal Validity

Single LLM backend on the headline benchmark. The 312-requirement benchmark uses Claude Sonnet 4.6 throughout. The multi-backend comparison in Section 5.4 adds a Haiku 4.5 datapoint on a 78-requirement subset; comparison with GPT-4 / Gemini / Llama-3 backends is recommended future work.

Limited retry budget. The verification loop terminates at $k = 3$ retries. A larger retry budget might push verification pass rate closer to 100 %.

7.3 External Validity

Three small reference apps. Each reference application is under 150 lines of Python. Scaling to industrial codebases (tens of thousands of lines, hundreds of endpoints) is not demonstrated.

Limited governance evidence for regulated domains. The shipped governance configuration contains generic redaction patterns (JIRA IDs, internal hostnames) rather than HIPAA/PCI-specific patterns. A revision with domain-specific patterns is recommended.

No head-to-head with EvoSuite/AthenaTest or commercial LLM-test tools. RAITG is not benchmarked against alternative test-generation tools on shared reference apps; such a comparison is recommended future work.

7.4 Conclusion Validity

No multiple-comparison correction. We report 8 pairwise metric comparisons in Table 4 without Bonferroni correction. With $\alpha =$

Table 10: Per-domain paired statistical comparisons. Source: tables/per_domain_significance.csv.

Domain	Metric	A vs B	<i>n</i>	Diff mean	Cohen’s <i>d</i>	Wilcoxon <i>p</i>	Sig. at 0.05
Commercial Web	Verify	Full vs Unverified	112	0.938	5.45	< 0.001	yes
Commercial Web	Verify	Full vs Ablation	112	0.232	0.63	0.0002	yes
Financial Services	Verify	Full vs Unverified	98	0.959	6.82	< 0.001	yes
Financial Services	Verify	Full vs Ablation	98	0.184	0.56	0.0008	yes
Healthcare	Verify	Full vs Unverified	102	0.971	8.08	< 0.001	yes
Healthcare	Verify	Full vs Ablation	102	0.235	0.70	< 0.001	yes

0.05 and 8 tests, expected spurious significant results would be 0.4 under the null; reported significant results (6 of 8) are far above this, so qualitative conclusions are robust.

Mutation testing’s equivalent-mutant problem. As documented in the Construct Validity subsection, our 82.99 % executable mutation score has a known upper bound: equivalent mutants (mutants that are semantically identical to the original) cannot be killed by any test and therefore impose a ceiling. The 33 surviving mutants in our experiment fall predominantly into equivalent or near-equivalent categories. We make no claim that a hypothetical perfect test suite could push our executable score to 100 %; the practical ceiling on this corpus is approximately 90 % after equivalent-mutant exclusion.

8 Conclusion

We present an empirical evaluation of the RAITG four-stage LLM-based test-generation pipeline on a 312-requirement, three-domain benchmark. The verification stage contributes 21.79 percentage points to verification pass rate (Cohen’s $d = 0.63$, $p < 0.001$). Coverage rises from 0 % (unverified, artifact of missing schema) to 100 % (full). A symbolic mutation indicator rises from 46.7 % to 59.5 %; explicitly disclosed as a weaker signal than executable mutation. A boundary-comprehensive baseline test suite attains an executable mutation score of 82.99 % on the three reference applications, exceeding the 60–75 % literature range for hand-authored PIT-style mutation on commodity codebases; per-operator analysis attributes the lift primarily to exact-boundary testing of numeric thresholds. A multi-backend comparison (Sonnet vs Haiku) reveals a 33× cost gap with complementary quality profiles, motivating tiered deployment. Per-domain analysis confirms the verification gain holds across commercial web, financial services, and healthcare.

We disclose four methodological limitations: symbolic-vs-executable mutation framing, literature-extrapolated manual baseline, format-handicapped unverified baseline, and the equivalent-mutant ceiling on executable mutation score.

8.1 Cost Analysis and Practical Deployability

A practical concern for LLM-based test generation is operational cost. We report the API-cost and wall-clock-time profile of the 312-requirement experiment to enable informed deployment decisions.

The total API cost of generating tests for 312 requirements across three domains was approximately USD \$7 in May 2026 pricing. The wall-clock speedup over a literature-cited human baseline is ~29×. We disclose that the ratio is partly arithmetic (comparing measured wall-clock against an extrapolated manual estimate rather than matched-author labour) and discuss this in Section 7.

Table 11: Per-requirement and total operational cost of the full RAITG pipeline (Anthropic Claude Sonnet 4.6, May 2026 API pricing). The reported figures are averages across the 312 requirements and include all three experimental conditions. Source: results/cost_profile.csv.

Metric	Per requirement	Total (312 reqs)
Input tokens (Stage 1–2)	~1,900	593,000
Output tokens (Stage 1–2)	~2,100	655,000
Verification rule executions (Stage 4)	14–18 rules	~5,000 invocations
Wall-clock (full pipeline)	74 sec	6.4 hours
Manual baseline (literature-cited)	35 min	184.1 hours (extrapolated)
Speedup	28×	28.7×
API cost (Claude Sonnet 4.6 rate)	USD \$0.022	USD \$6.83

8.2 Failure Mode Taxonomy

We hand-categorise the 73 verification failures (4.4 % of all generated tests) into five mutually exclusive failure modes. The taxonomy is useful both as a diagnostic for the rule engine and as a roadmap for future verification-rule additions.

Table 12: Hand-categorised failure-mode distribution across the 73 verification-failed test artefacts (out of 1,664 total generated tests in the full-RAITG condition). The largest single mode is “hallucinated precondition” (38 %), addressable by extending the Stage 4 rule engine with input-validity checks. Source: results/failure_modes.csv.

Failure mode	Count	Example
Hallucinated precondition	28 (38 %)	Test references field absent from API
Coverage-rule miss	19 (26 %)	Boundary condition not enumerated
Output-contract format error	14 (19 %)	Generated test lacks BDD scaffold
Redundancy with prior test	9 (12 %)	Near-duplicate of an earlier test
Logical inconsistency	3 (4 %)	Test asserts contradictory state

This taxonomy directly informs roadmap priorities: hallucinated preconditions and coverage misses together account for 64 % of verification failures and are addressable by Stage 4 rule extensions (input-validity checks, equivalence-class coverage rules). Format errors are addressable by tightening the output-contract template in Stage 2. The remaining 16 % (redundancy + logical inconsistency) require deeper LLM-level intervention.

Acknowledgments

The author used Anthropic’s Claude (a large language model) for drafting assistance and copy-editing of this manuscript. All empirical claims, dataset extraction, model training, and statistical analyses were generated by the author’s own scripts and verified against the on-disk artifacts.

Data availability. The primary dataset is permanently archived on Zenodo (*RAITG: Requirement-Aware Intelligent Test Generator*

Dataset, DOI: 10.5281/zenodo.20285104). Source code and analysis scripts are available at https://github.com/javvadivijayprasad/TestCaseGen_Reserach.

References

- [1] Leo Breiman. 2001. Random Forests. *Machine Learning* 45, 1 (2001), 5–32.
- [2] Tianqi Chen and Carlos Guestrin. 2016. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 785–794.